

virtspawn

REST API Reference

Complete reference for all 40+ API endpoints

Base URL: <http://localhost:8081/api/v1>

All responses are JSON | Errors return {"error": "message"}

GET

POST

DELETE

Content-Type: application/json

<https://github.com/ssahani/-virtspawn>

VM Lifecycle

Method	Endpoint	Description	Request Body
GET	/vms	List all VMs	-
GET	/vms/{name}	Get VM details	-
GET	/vms/{name}/xml	Get VM XML definition	-
POST	/vms	Create new VM	{name, vcpus, memory_mb, disk_gb}
POST	/vms/{name}/start	Start VM	-
POST	/vms/{name}/stop	Force stop VM	-
POST	/vms/{name}/shutdown	Graceful shutdown	-
POST	/vms/{name}/reboot	Reboot VM	-
POST	/vms/{name}/pause	Pause VM	-
POST	/vms/{name}/resume	Resume paused VM	-
DELETE	/vms/{name}	Delete VM	-
POST	/vms/{name}/clone	Clone VM	{new_name}
POST	/vms/{name}/rename	Rename VM (must be off)	{new_name}
POST	/vms/{name}/autostart/{bool}	Set autostart	-
POST	/vms/{name}/vcpus/{n}	Set vCPU count	-
POST	/vms/{name}/memory/{mb}	Set memory (MB)	-

Networks

Method	Endpoint	Description	Request Body
GET	/networks	List all networks	-
GET	/networks/{name}/xml	Get network XML	-
POST	/networks	Create network	{name, subnet, dhcp_start, dhcp_end}
POST	/networks/{name}/start	Start network	-
POST	/networks/{name}/stop	Stop network	-
POST	/networks/{name}/autostart/{bool}	Set autostart	-
DELETE	/networks/{name}	Delete network	-

Storage

Method	Endpoint	Description	Request Body
GET	/storage/pools	List storage pools	-
POST	/storage/pools	Create pool	{name, pool_type, target_path}
DELETE	/storage/pools/{name}	Delete pool	-
GET	/storage/pools/{name}/xml	Get pool XML	-
GET	/storage/pools/{p}/volumes	List volumes in pool	-
POST	/storage/pools/{p}/volumes	Create volume	{name, capacity_gb, format}
DELETE	/storage/pools/{p}/volumes/{v}	Delete volume	-
POST	/storage/pools/{p}/volumes/{v}/resize	Resize volume	{capacity_gb}
POST	/storage/pools/{p}/volumes/{v}/clone	Clone volume	{new_name}

Snapshots

Method	Endpoint	Description	Request Body
GET	/snapshots	List all snapshots	-
GET	/vms/{name}/snapshots	List snapshots for VM	-
POST	/vms/{name}/snapshots	Create snapshot	{name, description}
POST	/vms/{n}/snapshots/{s}/revert	Revert to snapshot	-
DELETE	/vms/{n}/snapshots/{s}	Delete snapshot	-

Metrics & Node

Method	Endpoint	Description	Request Body
GET	/metrics	All VM metrics	-
GET	/metrics/{name}	Single VM metrics	-
GET	/node	Host node info	-
GET	/health	Health check	-
GET	/templates	List VM templates	-

WebSocket

Method	Endpoint	Description	Request Body
WS	/ws/v1/watch	Real-time VM state changes	subscribe
WS	/ws/v1/console/{name}	Serial console I/O	bidirectional
WS	/ws/v1/vnc/{name}	VNC proxy (binary)	bidirectional

Metrics response: cpu_time_ns, memory_total_mb, memory_used_mb, memory_pct, disk/net I/O

Guest Agent & Boot

Method	Endpoint	Description	Request Body
GET	/vms/{name}/interfaces	Guest IP addresses (via agent)	-
GET	/vms/{name}/hostname	Guest hostname	-
GET	/vms/{name}/boot	Get boot configuration	-
POST	/vms/{name}/boot	Set boot order	<code>{devices: ['hd*', 'cdrom']}</code>

CD-ROM, Save & Migration

Method	Endpoint	Description	Request Body
POST	/vms/{name}/cdrom/insert	Insert CD-ROM ISO	<code>{iso_path, target}</code>
POST	/vms/{name}/cdrom/eject/{target}	Eject CD-ROM	-
POST	/vms/{name}/managed-save	Managed save (hibernate)	-
DELETE	/vms/{name}/managed-save	Remove saved state	-
GET	/vms/{name}/managed-save/status	Check if save exists	-
POST	/vms/{name}/migrate	Migrate VM	<code>{dest_url, live}</code>

Interface

POST	/vms/{name}/balloon/{mb}	Memory balloon	-
------	--------------------------	----------------	---

Method	Endpoint	Description	Request Body
GET	/capabilities	Hypervisor capabilities	-
GET	/sysinfo	System BIOS/SMBIOS info (XML)	-
GET	/devices	List node devices	<code>?capability=pci net usb</code>
GET	/devices/{name}	Get device XML	-
GET	/nwfilters	List network filters	-
DELETE	/nwfilters/{name}	Delete network filter	-
GET	/secrets	List secrets	-
DELETE	/secrets/{uuid}	Delete secret	-

Backup Operations

Method	Endpoint	Description	Request Body
GET	/backups	List all backups with status/size	-
POST	/backups	Trigger backup	{vm_name, with_disks, incremental, retain}
GET	/backups/{id}/status	Get backup progress & status	-
POST	/backups/{id}/verify	Verify SHA-256 checksums	-
GET	/backups/{id}/download	Download backup as tar.gz	-
POST	/backups/restore	Restore from backup	{backup_id}
DELETE	/backups/{id}	Delete a backup	-

Schedule Management

Method	Endpoint	Description	Request Body
GET	/backups/schedule	Get systemd timer status	-
POST	/backups/schedule	Enable/disable timer	{enabled: true/false}

Backup Examples

```
# Backup all VMs
$ curl -X POST localhost8081/api/v1/backups \
-d '{"retain": 7}'

# Per-VM with disks
$ curl -X POST localhost8081/api/v1/backups \
-d '{"vm_name": "myvm", "with_disks": true}'
```

```
# Verify checksums
$ curl -X POST localhost8081/api/v1/backups/20260324-020000/verify

# Download as tar.gz
$ curl -O localhost8081/api/v1/backups/20260324-020000/download

# Enable scheduled backups
$ curl -X POST localhost8081/api/v1/backups/schedule \
-d '{"enabled": true}'
```


200	OK	Successful GET/POST/DELETE
404	Not Found	VM, network, pool, volume, or snapshot not found
500	Internal Server Error	Libvirt operation failed, connection error

Input Validation

VM/resource names: alphanumeric, dash, underscore, dot (1-64 chars)

vCPUs: 1-256

Memory: 64 MB - 1 TB (1,048,576 MB)

Disk size: 1 GB - 10 TB (10,240 GB)

Migration URI: must start with qemu://, qemu+ssh://, qemu+tcp://, qemu+tls://, qemu+unix//

Volume capacity: must be > 0, checked_mul for overflow protection

ISO paths: must be absolute, canonicalized, and point to existing file

Network IPs: validated as IPv4 format, subnet as 3-octet prefix

Memory balloon: validated against 64 MB - 1 TB range

Security: No CORS headers (same-origin only) | XML-escaped inputs | PTY path validated | Audit logged

